



Paradigmas de Programación (EIF-400) Caso de FP-OOP en Java Parte-B

CARLOS LORÍA-SÁENZ LORACARLOS@GMAIL.COM

AGOSTO-SETIEMBRE 2025

EIF/UNA

Objetivos

2

- ▶ Operaciones de reducción y agregación en FP-OOP

Objetivos Específicos

3

- ▶ Combinadores de agregación (colectores)
- ▶ Comparadores

Collector: Casos de Uso

4

- ▶ Tomar una colección de datos y reducirla a un valor (lo mismo que pretende `reduce`)
- ▶ Son “reducers” para caso comunes (aka agregaciones)
 - ▶ Convertir un stream de personas en una lista de personas (si es finito)
 - ▶ El máximo (mínimo), suma, promedio de atributos de objetos en una colección. **Ejemplo** suma de las edades de las personas en una lista. O la máxima edad.
 - ▶ Clasificar(agrupar) los objetos de una colección por atributo en forma booleana (binaria): los que lo tienen contra los que no lo tienen. **Ejemplo**: Agrupar hombres y mujeres de una lista de personas
 - ▶ Clasificar (agrupar) (`GroupBy`) según los posibles valores de un atributo (key). **Ejemplo**: Separar por provincia a una lista de personas

Stream y collect

5

- ▶ Stream tiene un método `collect` que recibe un `Collector` para realizar la reducción y retorna el resultado correspondiente

java.util.stream

Interface Stream<T>

Type Parameters:

T - the type of the stream elements

returns a Builder for a Stream.

<R,A> R

`collect(Collector<? super T,A,R> collector)`

Performs a **mutable reduction** operation on the elements of this stream using a `Collector`.

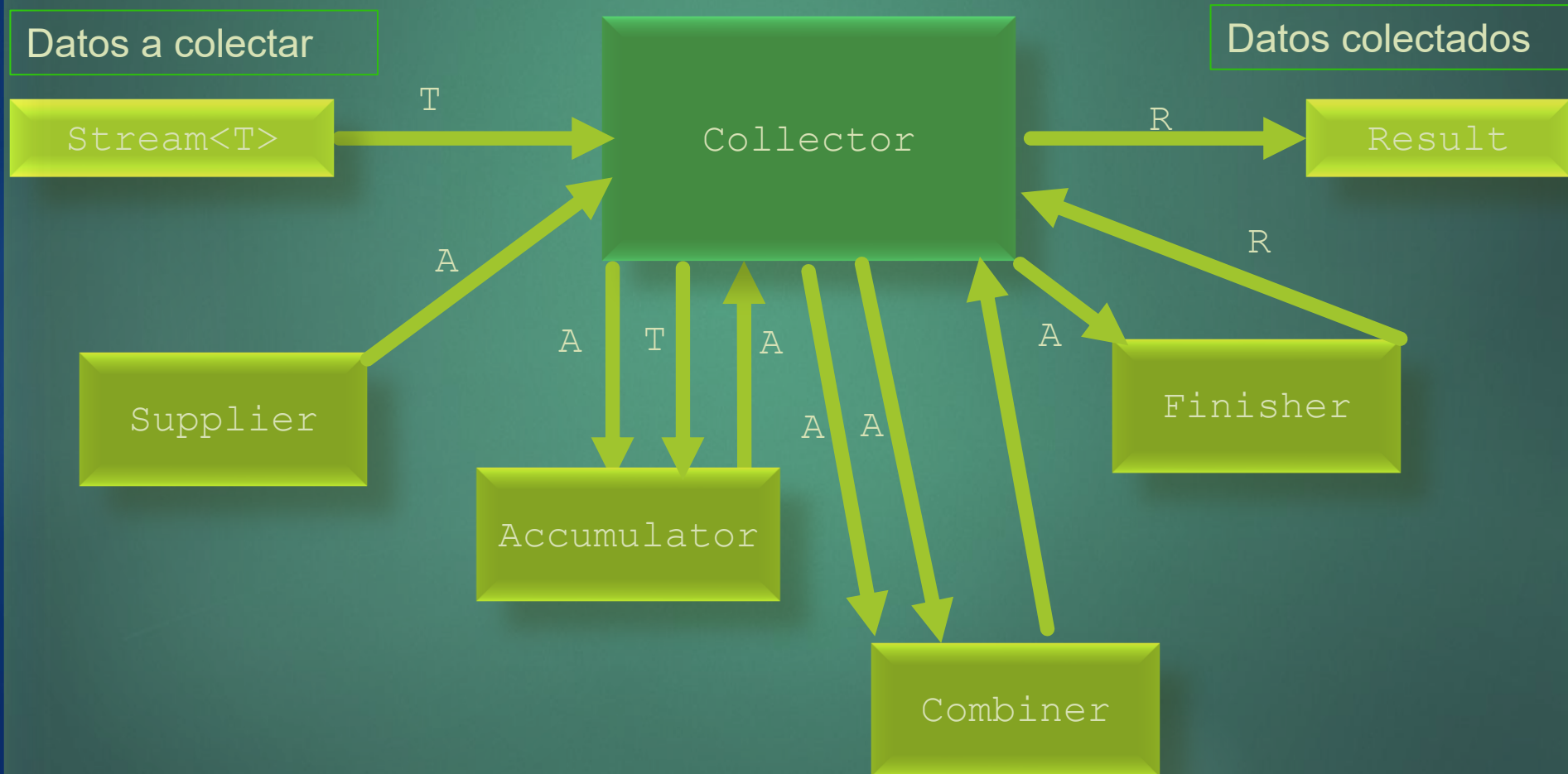
<R> R

`collect(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R,R> combiner)`

Performs a **mutable reduction** operation on the elements of this stream.

Collector<T, A, R>

6



Collectors como Factory

- ▶ `Collector` tiene métodos para crear `Collectors` (es un `Factory`)
- ▶ Por ejemplo: `Collectors.toList` retorna un `Collector` que convierte un `Stream` en una `Lista`

```
jshell> var personas = Arrays.asList("Ana", "Pedro", "Maria", "Jose", "Jesus")
personas ==> [Ana, Pedro, Maria, Jose, Jesus]

jshell> personas.stream().filter( name -> name.startsWith("J") ).collect(Collectors.toList())
$62 ==> [Jose, Jesus]

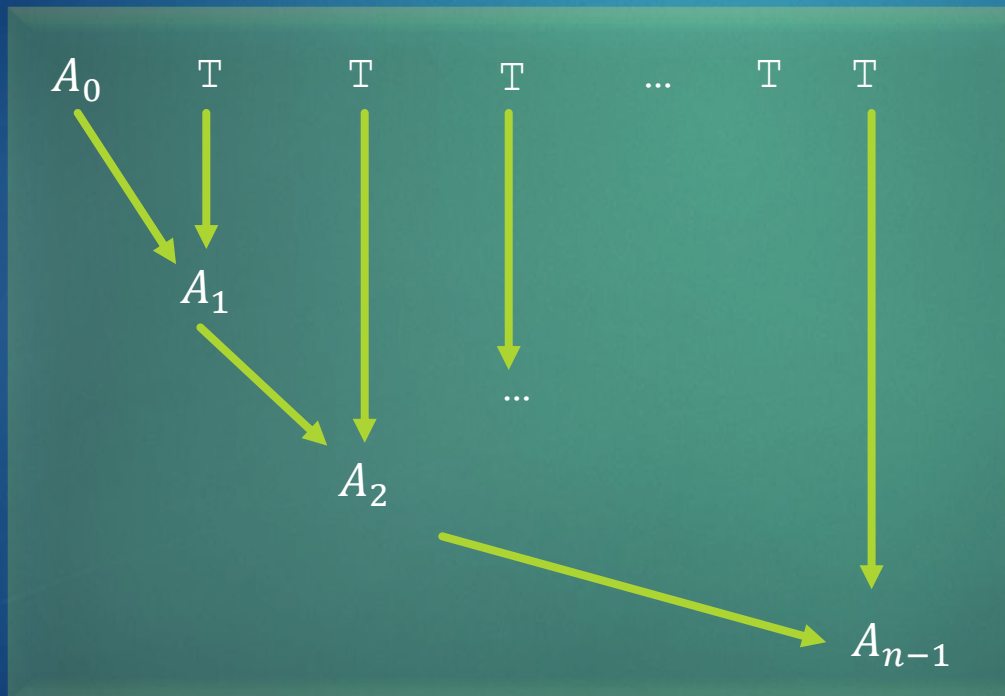
jshell> personas.stream().filter( name -> name.endsWith("a") ).collect(Collectors.toList())
$63 ==> [Ana, Maria]

jshell> |
```

Propiedades

8

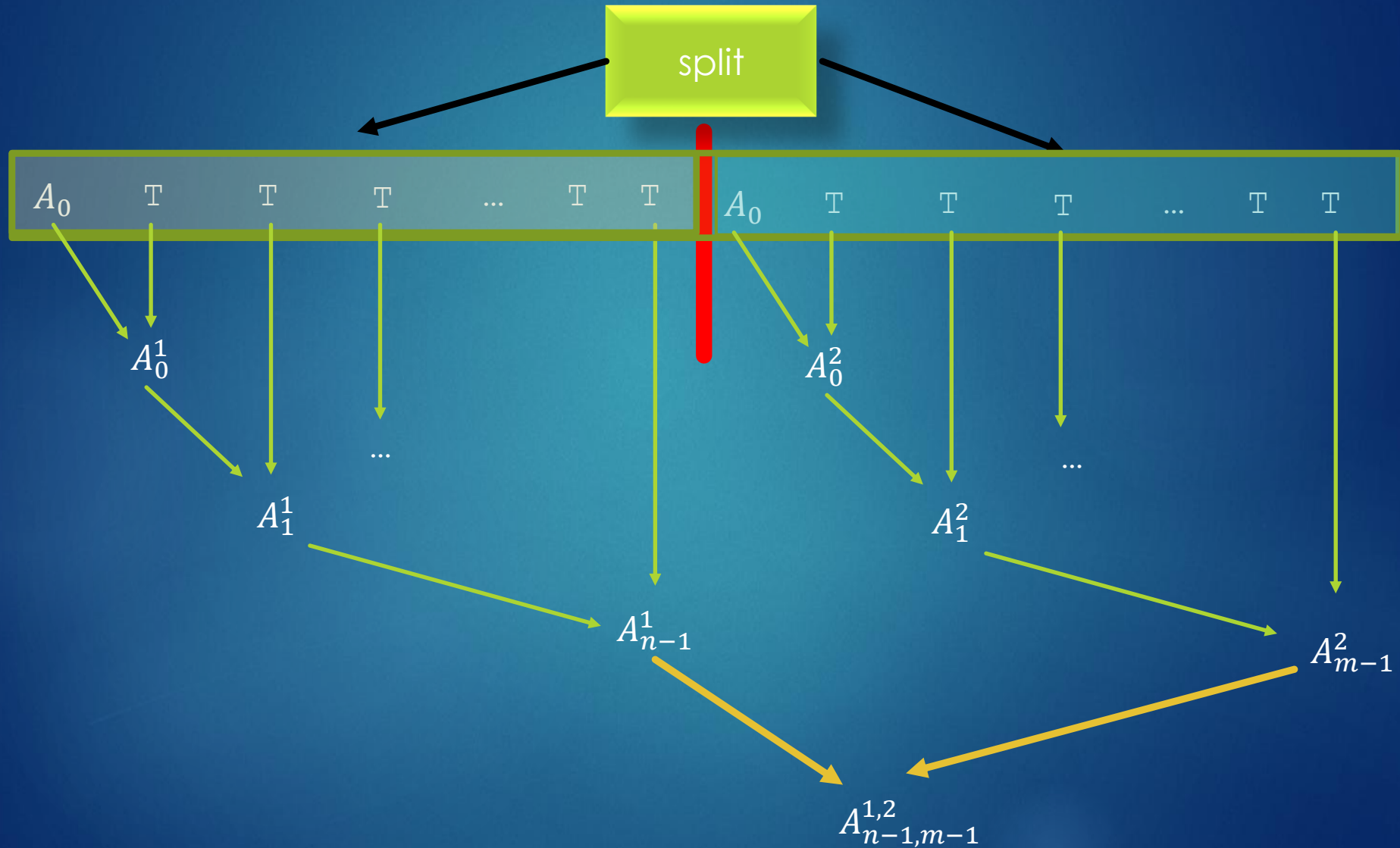
- ▶ Sea $A_0 = \text{supplier}()$
- ▶ `combiner` debe ser asociativo
- ▶ Y A_0 debe ser neutro de `combiner`



Una reducción en serie

En paralelo

9



Ejemplos Team

10

```
jshell> record Team(String name, int titles){}  
|   created record Team  
  
jshell> Team team(String name, int titles){return new Team(name, titles);}  
|   created method team(String,int)
```

```
jshell> var teams = Arrays.asList(team("csh", 29), team("lda", 30), team("sap", 36), team("csh", 4), team("mpz", 1), team("put", 1), team("sc", 1))  
teams ==> [Team[name=csh, titles=29], Team[name=lda, titles ... , Team[name=sc, titles=1]]
```

```
jshell> teams.stream().collect(Collectors.counting())  
$10 ==> 7  
  
jshell> teams.stream().collect(Collectors.summingInt( t -> t.titles()))  
$11 ==> 102
```

```
jshell> teams.stream().collect(Collectors.averagingInt( t -> t.titles() ) )  
$15 ==> 14.571428571428571  
  
jshell> 102.0/7  
$16 ==> 14.571428571428571
```

Extractor de
un entero
de un Team

Team ...

11

```
jshell> teams.stream().collect(Collectors.maxBy( (t1, t2) -> t1.titles() - t2.titles()))  
$17 ==> Optional[Team[name=sap, titles=36]]
```

```
jshell> teams.stream().collect(Collectors.minBy( (t1, t2) -> t1.titles() - t2.titles()))  
$18 ==> Optional[Team[name=mpz, titles=1]]
```

```
jshell> println("team,titles\n" + teams.stream().map(t -> teamToCsv(t)).collect(Collectors.joining( "\n")))  
team,titles  
csh,29  
lda,30  
sap,36  
csh,4  
mpz,1  
put,1  
sc,1  
jshell>
```

Team ...

12

```
jshell> String teamToCsv(Team t){return String.format("%s,%s", t.name(), t.titles());}  
| created method teamToCsv(Team)
```

```
jshell> println("team,titles\n" + teams.stream().map(t -> teamToCsv(t)).collect(Collectors.joining( "\n")))  
team,titles  
csh,29  
lda,30  
sap,36  
csh,4  
mpz,1  
put,1  
sc,1
```

Team ...

13

```
jshell> var teamsByFirstLetter = teams.stream().collect(Collectors.groupingBy( t -> t.name().charAt(0)))
teamsByFirstLetter ==> {p=[Team[name=put, titles=1]], s=[Team[name=sap, ... Team[name=mpz, titles=1]]}

jshell> teamsByFirstLetter.get('c')
$8 ==> [Team[name=csh, titles=29], Team[name=csh, titles=4]]

jshell> |
```

Comparator<T>

14

- ▶ Interfaz funcional para definir un comparador entre dos objetos de tipo `T`
- ▶ Puede entonces ser reemplazada por una lambda
- ▶ También es un Factory de comparadores
- ▶ El método abstracto es `int compare(T a, T b)`
- ▶ Convención:
 - ▶ Si retorna negativo “gana” `a` (va primero `a`)
 - ▶ Si retorna cero empatan
 - ▶ Si retorna positivo “gana” `b` (va primero `b`)

Un comparador

- ▶ Se puede usar cualquier lambda apropiada
- ▶ Se puede también usar alguno de los métodos estáticos de `Comparator`
- ▶ Por ejemplo: Un comparador para un atributo entero:
- ▶ Ira “primero” el que tenga menos títulos (de menor a mayor)

```
jshell> var comparatorByTitles = Comparator.comparingInt( (Team a) -> a.titles() )  
comparatorByTitles ==> java.util.Comparator$$Lambda$21/0x00000000800c5b6a@7e774085
```

Ejemplo completo

16

```
jshell> record Team(String name, int titles){}  
|   created record Team
```

```
jshell> var csh = new Team("CSH", 29)  
csh ==> Team[name=CSH, titles=29]
```

```
jshell> var lda = new Team("LDA", 30)  
lda ==> Team[name=LDA, titles=30]
```

```
jshell> var comparatorByTitles = Comparator.comparingInt( (Team a) -> a.titles() )  
comparatorByTitles ==> java.util.Comparator$$Lambda$21/0x0000000800c5b6a0@7e774085
```

```
jshell> comparatorByTitles.compare(csh, lda)  
$5 ==> -1
```

Teams sort ...

17

```
jshell> teams.stream().sorted(comparatorByTitles).map( Team::name).toList()  
$15 ==> [mpz, put, sc, csh, csh, lda, sap]  
  
jshell> |
```

Ejemplo: Creando un Collector

- ▶ Ver demo `myCollector.java`

```
PP:java -cp classes com.eif400.demo.MyCollector  
[abc, d, efgh,      ]  
17  
  
PP:
```

Ejemplo

19

► Ver demo CollectorsAggregations

```
PP:java -cp classes com.eif400.demo.Main
*** Agregations using Collectors ***
*** Testing java FP Core Knowledge ***

---ALL PERSONS---
Person[name=Gabriels, Hanna, age=38, country=CRC]
Person[name=Trump, Donald, age=73, country=USA]
Person[name=Chu, Pika, age=25, country=JPN]
Person[name=Centeno, Walter (King Pate), age=46, country=CRC]
Person[name=Klopf, Juergen, age=79, country=BRD]
Person[name=Merkle, Angela, age=67, country=BRD]
Person[name=Putin, Vladimir, age=66, country=RUS]
Person[name=Alvarado, Charlie, age=44, country=CRC]
Person[name=Jinpig, Xi, age=68, country=CHN]
Person[name=Mbappe, Kilian, age=22, country=FRC]
Person[name=Oh, Yu Gi, age=123, country=JPN]
Person[name=Cheen, Charlie, age=52, country=USA]
Person[name=Johansson, Scarlett, age=36, country=USA]
Person[name=Biden, Joe, age=77, country=USA]

---ALL COUNTRIES---
Country: CRC (CRC)
Country: JPN (JPN)
Country: USA (USA)
Country: RUS (RUS)
Country: ARG (ARG)
Country: BRZ (BRZ)
Country: CHN (CHN)
Country: FRC (FRC)
Country: AFG (AFG)
Country: CUB (CUB)
Country: BRD (BRD)
```

Salida fue truncada